

应用层常见协议

HTTP HTTPS DNS FTP SMTP

七层网络和5层体系

自顶向下分别是

- **应用层**：最靠近用户的层，负责处理特定的应用程序细节。这一层提供了网络服务与用户应用软件之间的接口。例如，Web 浏览器、FTP 客户端和服务端、电子邮件客户端等。
- **表示层**：确保从一个系统发送的信息可以被另一个系统的应用层读取。它负责数据的转换、压缩和加密。例如，确保数据从一种编码格式转换为另一种，如 ASCII 到 EBCDIC。
- **会话层**：管理用户的会话，控制网络上两节点间的对话和数据交换的管理。它负责建立、维护和终止会话。例如，建立一个会话令牌，以便在网络上的两个节点之间传递。
- **传输层**：提供端到端的通信服务，保证数据的完整性和正确顺序。这一层包括 TCP 和 UDP 等。
- **网络层**：负责在多个网络之间进行数据传输，确保数据能够在复杂的网络结构中找到从源到目的地的最佳路径。这层使用的是 IP (Internet Protocol) 协议。
- **数据链路层**：在物理连接中提供可靠的传输，负责建立和维护两个相邻节点间的链路。包括帧同步、MAC (媒体访问控制)。
- **物理层**：负责在物理媒介上实现原始的数据传输，比如电缆、光纤和无线信号传输。涉及的内容包括电压、接口、引脚、电缆的规格和传输速率等。

5层体系是应用层 传输层 网络层 数据链路层 物理层

数据链路层作用

在物理连接中提供可靠的传输，负责建立和维护两个相邻节点间的链路。包括帧同步、MAC (媒体访问控制)。

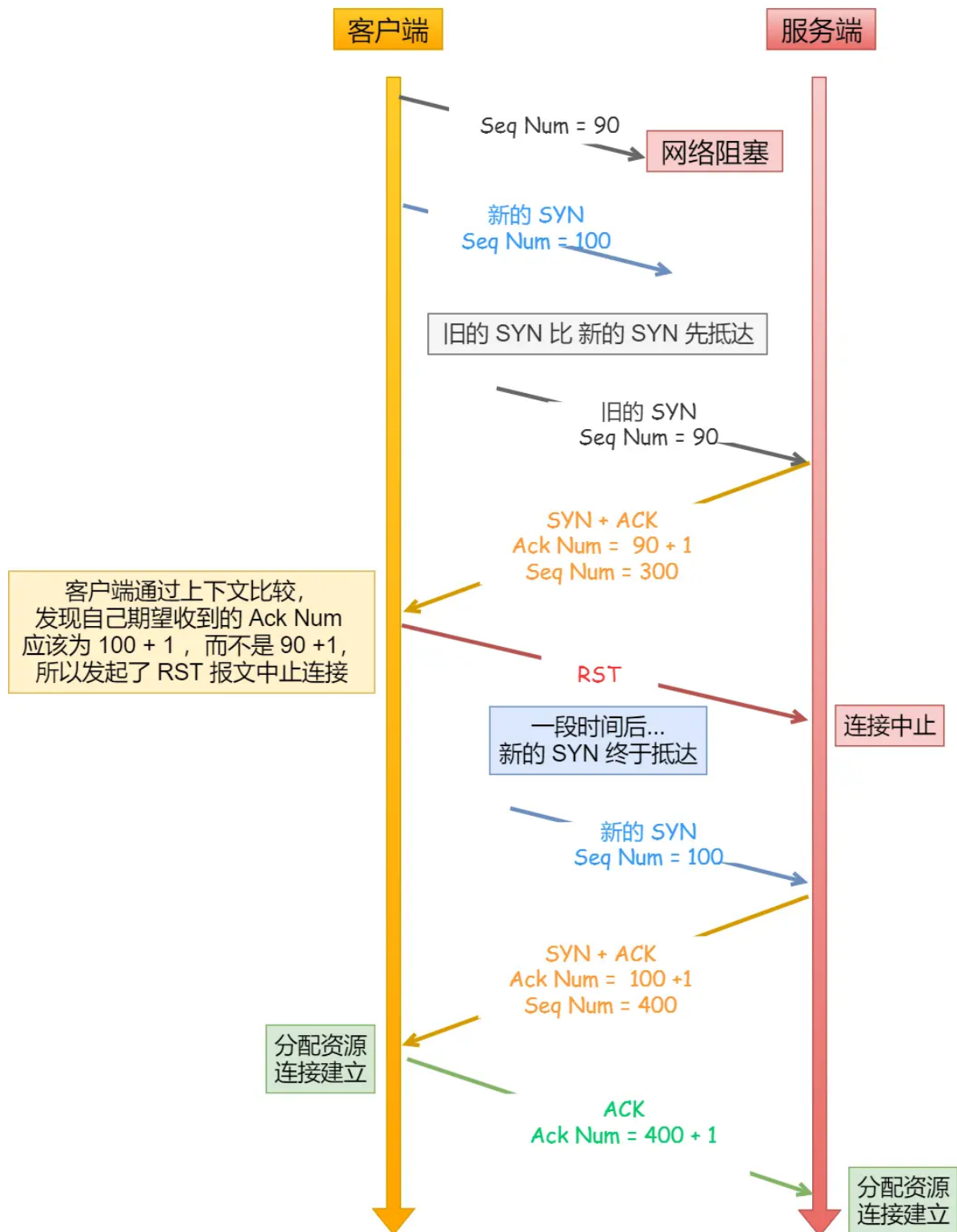
tcp为什么是三次握手

三次握手的首要原因是为了防止旧的重复连接初始化造成混乱。

我们考虑一个场景，客户端先发送了 SYN (seq = 90) 报文，然后客户端宕机了，而且这个 SYN 报文还被网络阻塞了，服务端并没有收到，接着客户端重启后，又重新向服务端建立连接，发送了 SYN (seq = 100) 报文

看看三次握手是如何阻止历史连接的：

三次握手 避免历史连接



客户端连续发送多次 SYN (都是同一个四元组) 建立连接的报文, 在**网络拥堵**情况下:

- 一个「旧 SYN 报文」比「最新的 SYN」报文早到达了服务端, 那么此时服务端就会回一个 **SYN + ACK** 报文给客户端, 此报文中的确认号是 $90 + 1$ 。
- 客户端收到后, 发现自己期望收到的确认号应该是 $100 + 1$, 而不是 $90 + 1$, 于是就会回 RST 报文。
- 服务端收到 RST 报文后, 就会释放连接。
- 后续最新的 SYN 抵达了服务端后, 客户端与服务端就可以正常的完成三次握手了。

上述中的「旧 SYN 报文」称为历史连接，TCP 使用三次握手建立连接的最主要原因就是防止「历史连接」初始化了连接。

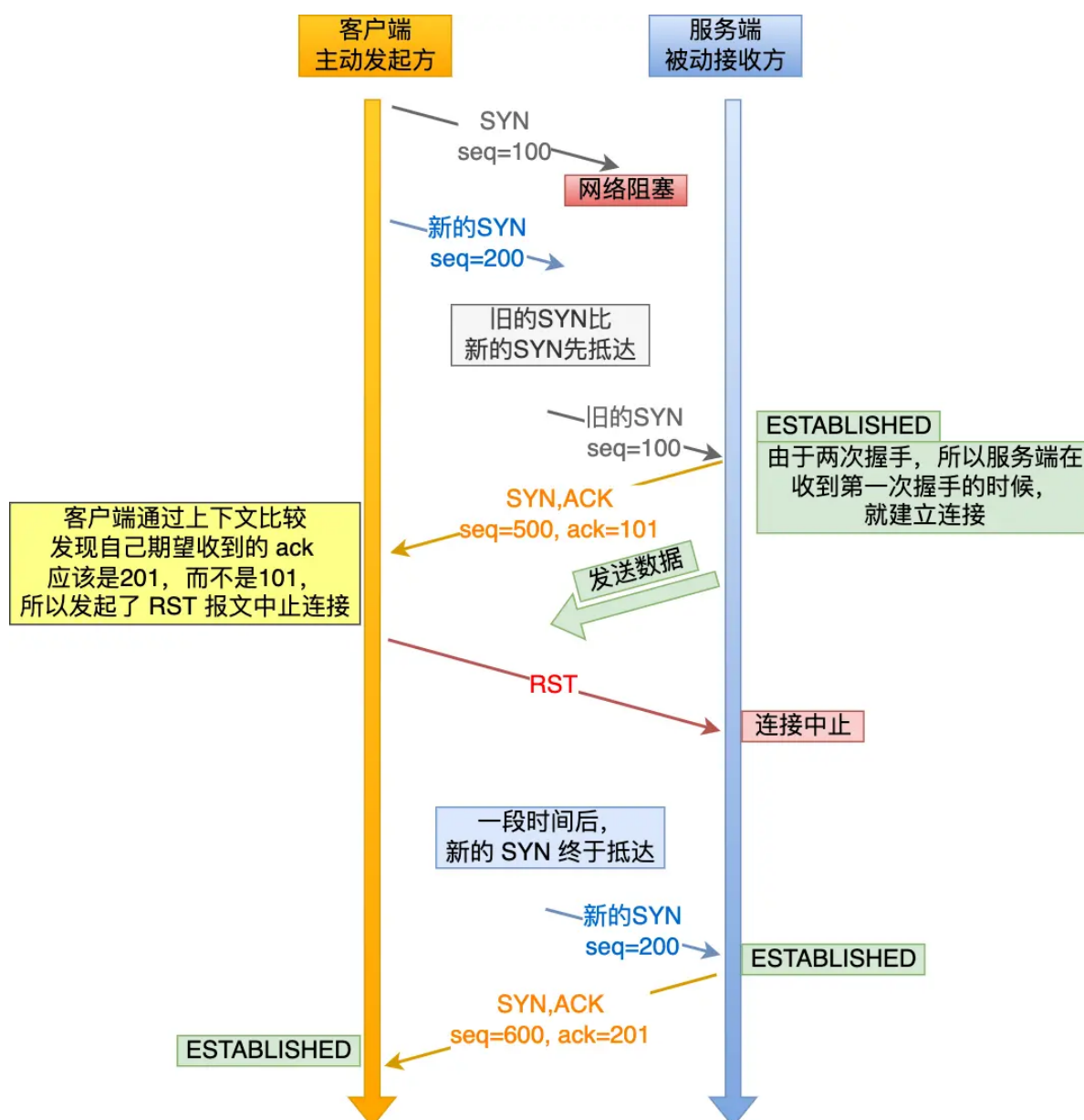
有很多人问，如果服务端在收到 RST 报文之前，先收到了「新 SYN 报文」，也就是服务端收到客户端报文的顺序是：「旧 SYN 报文」->「新 SYN 报文」，此时会发生什么？

当服务端第一次收到 SYN 报文，也就是收到「旧 SYN 报文」时，就会回复 SYN + ACK 报文给客户端，此报文中的确认号是 91 (90+1)。

然后这时再收到「新 SYN 报文」时，就会回 [Challenge Ack \(opens new window\)](#) 报文给客户端，**这个 ack 报文并不是确认收到「新 SYN 报文」的，而是上一次的 ack 确认号**，也就是 91 (90+1)。所以客户端收到此 ACK 报文时，发现自己期望收到的确认号应该是 101，而不是 91，于是就会回 RST 报文。

如果是两次握手连接，就无法阻止历史连接，因为在两次握手的情况下，服务端没有中间状态给客户端来阻止历史连接，导致服务端可能建立一个历史连接，造成资源浪费。

在两次握手的情况下，服务端在收到 SYN 报文后，就进入 ESTABLISHED 状态，意味着这时可以给对方发送数据，但是客户端此时还没有进入 ESTABLISHED 状态，假设这次是历史连接，客户端判断到此次连接为历史连接，那么就会回 RST 报文来断开连接，而服务端在第一次握手的时候就进入 ESTABLISHED 状态，所以它可以发送数据的，但是它并不知道这个是历史连接，它只有在收到 RST 报文后，才会断开连接。



可以看到，如果采用两次握手建立 TCP 连接的场景下，服务端在向客户端发送数据前，并没有阻止掉历史连接，导致服务端建立了一个历史连接，又白白发送了数据，妥妥地浪费了服务端的资源。

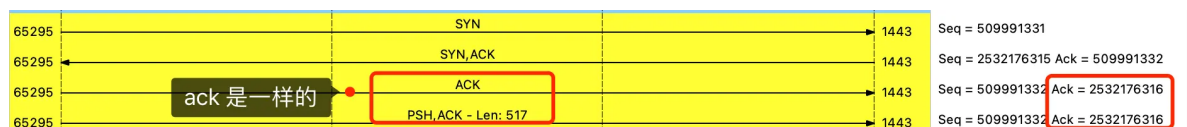
因此，要解决这种现象，最好就是在服务端发送数据前，也就是建立连接之前，要阻止掉历史连接，这样就不会造成资源浪费，而要实现这个功能，就需要三次握手。

所以，TCP 使用三次握手建立连接的最主要原因是防止「历史连接」初始化了连接。

TIP

有人问：客户端发送三次握手（ack 报文）后就可以发送数据了，而被动方此时还是 syn_received 状态，如果 ack 丢了，那客户端发的数据是不是也白白浪费了？

不是的，即使服务端还是在 syn_received 状态，收到了客户端发送的数据，还是可以建立连接的，并且还可以正常收到这个数据包。这是因为数据报文中是有 ack 标识位，也有确认号，这个确认号就是确认收到了第二次握手。如下图：



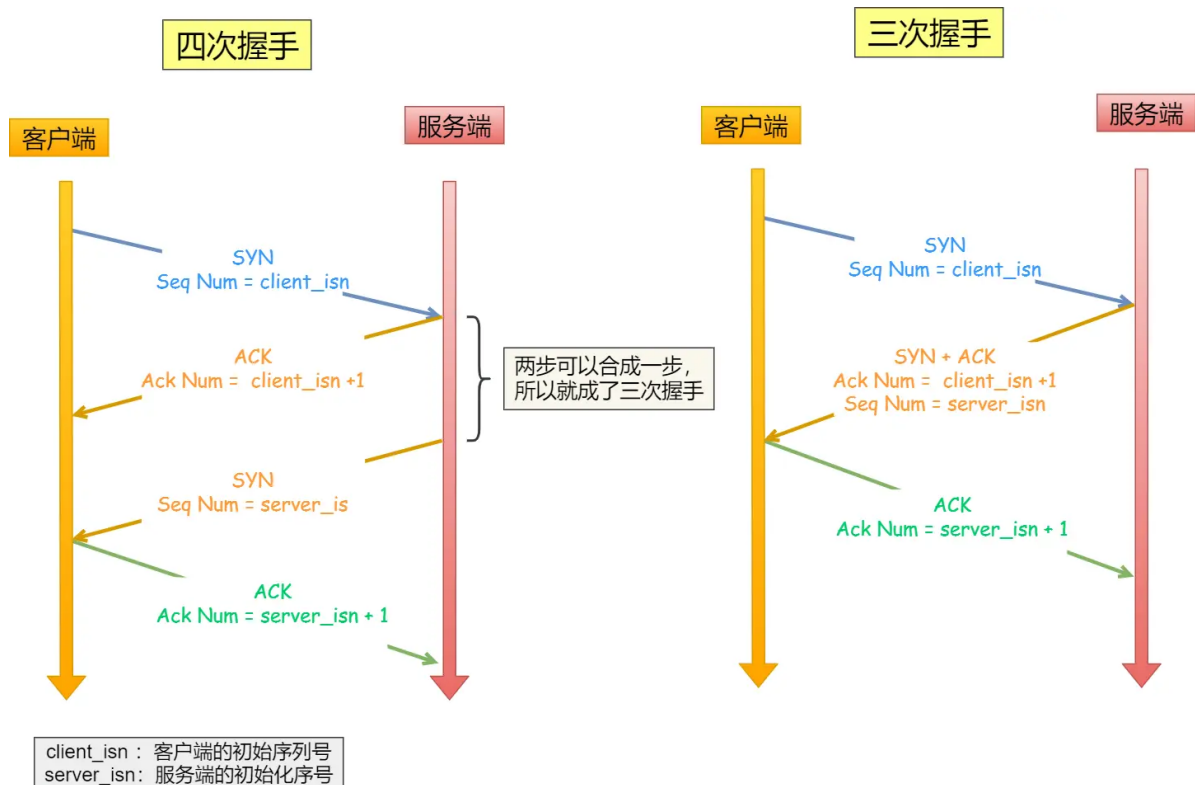
所以，服务端收到这个数据报文，是可以正常建立连接的，然后就可以正常接收这个数据包了。

原因二：同步双方初始序列号

TCP 协议的通信双方，都必须维护一个「序列号」，序列号是可靠传输的一个关键因素，它的作用：

- 接收方可以去除重复的数据；
- 接收方可以根据数据包的序列号按序接收；
- 可以标识发送出去的数据包中，哪些是已经被对方收到的（通过 ACK 报文中的序列号知道）；

可见，序列号在 TCP 连接中占据着非常重要的作用，所以当客户端发送携带「初始序列号」的 SYN 报文的时候，需要服务端回一个 ACK 应答报文，表示客户端的 SYN 报文已被服务端成功接收，那当服务端发送「初始序列号」给客户端的时候，依然也要得到客户端的应答回应，这样一来一回，才能确保双方的初始序列号能被可靠的同步。



四次握手其实也能够可靠的同步双方的初始化序号，但由于第二步和第三步可以优化成一步，所以就成了「三次握手」。

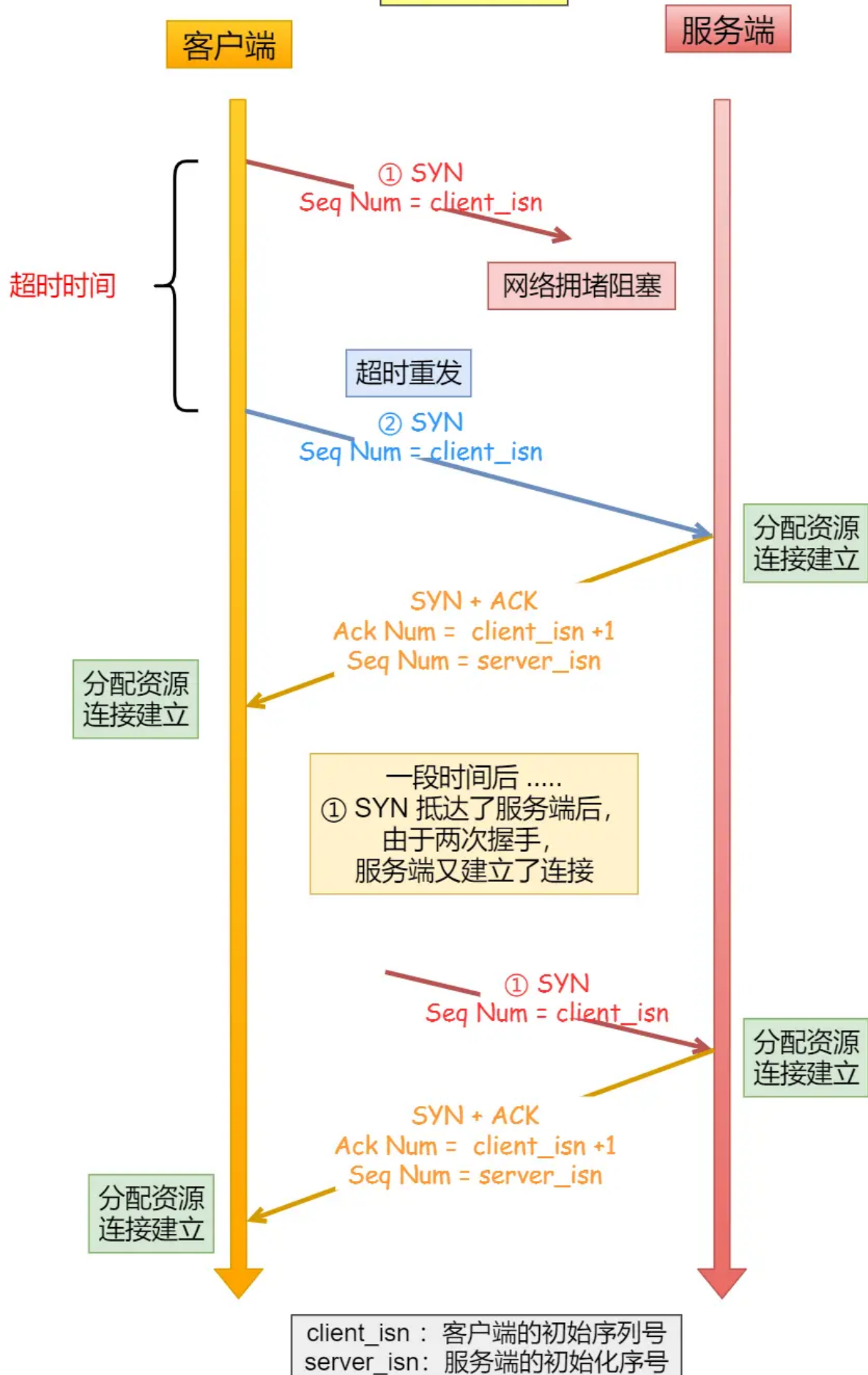
而两次握手只保证了一方的初始序列号能被对方成功接收，没办法保证双方的初始序列号都能被确认接收。

原因三：避免资源浪费

如果只有「两次握手」，当客户端发生的 SYN 报文在网络中阻塞，客户端没有接收到 ACK 报文，就会重新发送 SYN，**由于没有第三次握手，服务端不清楚客户端是否收到了自己回复的 ACK 报文，所以服务端每收到一个 SYN 就只能先主动建立一个连接**，这会造成什么情况呢？

如果客户端发送的 SYN 报文在网络中阻塞了，重复发送多次 SYN 报文，那么服务端在收到请求后就会**建立多个冗余的无效链接，造成不必要的资源浪费。**

两次握手



即两次握手会造成消息滞留情况下，服务端重复接受无用的连接请求 SYN 报文，而造成重复分配资源。

TIP

很多人问，两次握手不是也可以根据上下文信息丢弃 syn 历史报文吗？

我这里两次握手是假设「由于没有第三次握手，服务端不清楚客户端是否收到了自己发送的建立连接的 ACK 确认报文，所以每收到一个 SYN 就只能先主动建立一个连接」这个场景。

当然你要实现成类似三次握手那样，根据上下文丢弃 syn 历史报文也是可以的，两次握手没有具体的实现，怎么假设都行。

小结

TCP 建立连接时，通过三次握手**能防止历史连接的建立，能减少双方不必要的资源开销，能帮助双方同步初始化序列号**。序列号能够保证数据包不重复、不丢弃和按序传输。

不使用「两次握手」和「四次握手」的原因：

- 「两次握手」：无法防止历史连接的建立，会造成双方资源的浪费，也无法可靠的同步双方序列号；
- 「四次握手」：三次握手就已经理论上最少可靠连接建立，所以不需要使用更多的通信次数。

tcp粘包和拆包

粘包：多条消息被合并到一个 TCP 数据包中发送，接收方一次 `recv` 可能读到多条消息的数据。

主要有三类：

1. 发送端原因

- 应用层多次调用 `send()` 发送的小数据量，TCP 可能会把它们缓冲起来一次性发出（Nagle 算法）。
- TCP 为了提高网络利用率，会把多个小包合并成一个大包再发。

2. 接收端原因

- 接收方的 `recv()` 读取缓冲区时，可能一次读到了多个包，或者只读到一个包的一部分。
- 网络延迟和 MTU 限制导致数据分段。

3. 协议特性原因

- TCP 是流式的，没有“消息”的概念，只有连续的字节流，消息的边界要应用层自己定义。

拆包：一条完整的消息被拆成多个 TCP 数据包发送，接收方一次 `recv` 只能读到消息的一部分，需要多次读取才能拼完整。

报文最大生存生存时间

MSL

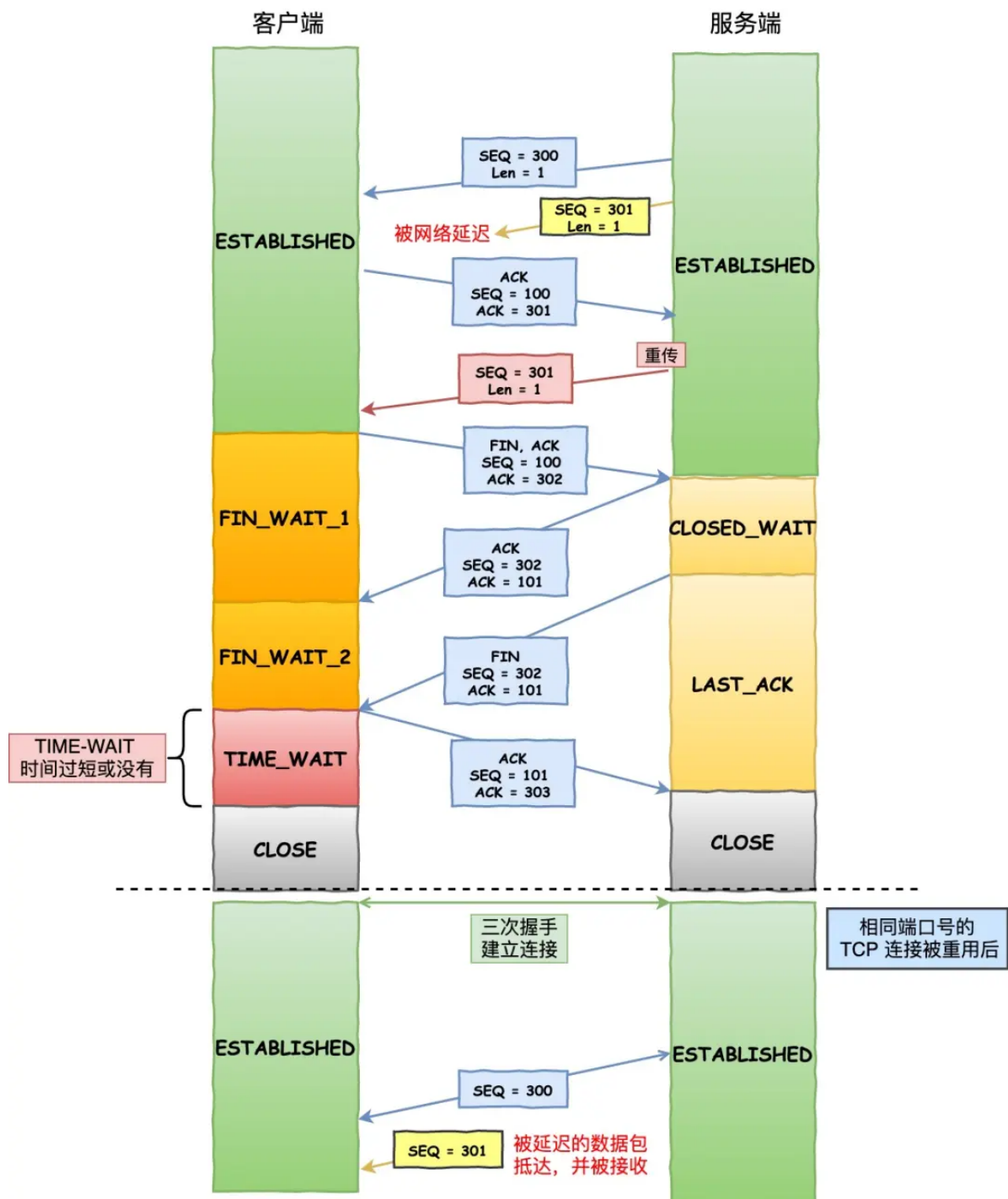
为什么需要time-wait状态

需要 TIME-WAIT 状态，主要是两个原因：

- 防止历史连接中的数据，被后面相同四元组的连接错误的接收；
- 保证「被动关闭连接」的一方，能被正确的关闭；

原因一：防止历史连接中的数据，被后面相同四元组的连接错误的接收

序列号和初始化序列号并不是无限递增的，会发生回绕为初始值的情况，这意味着无法根据序列号来判断新老数据。假设 TIME-WAIT 没有等待时间或时间过短，被延迟的数据包抵达后会发生什么呢？



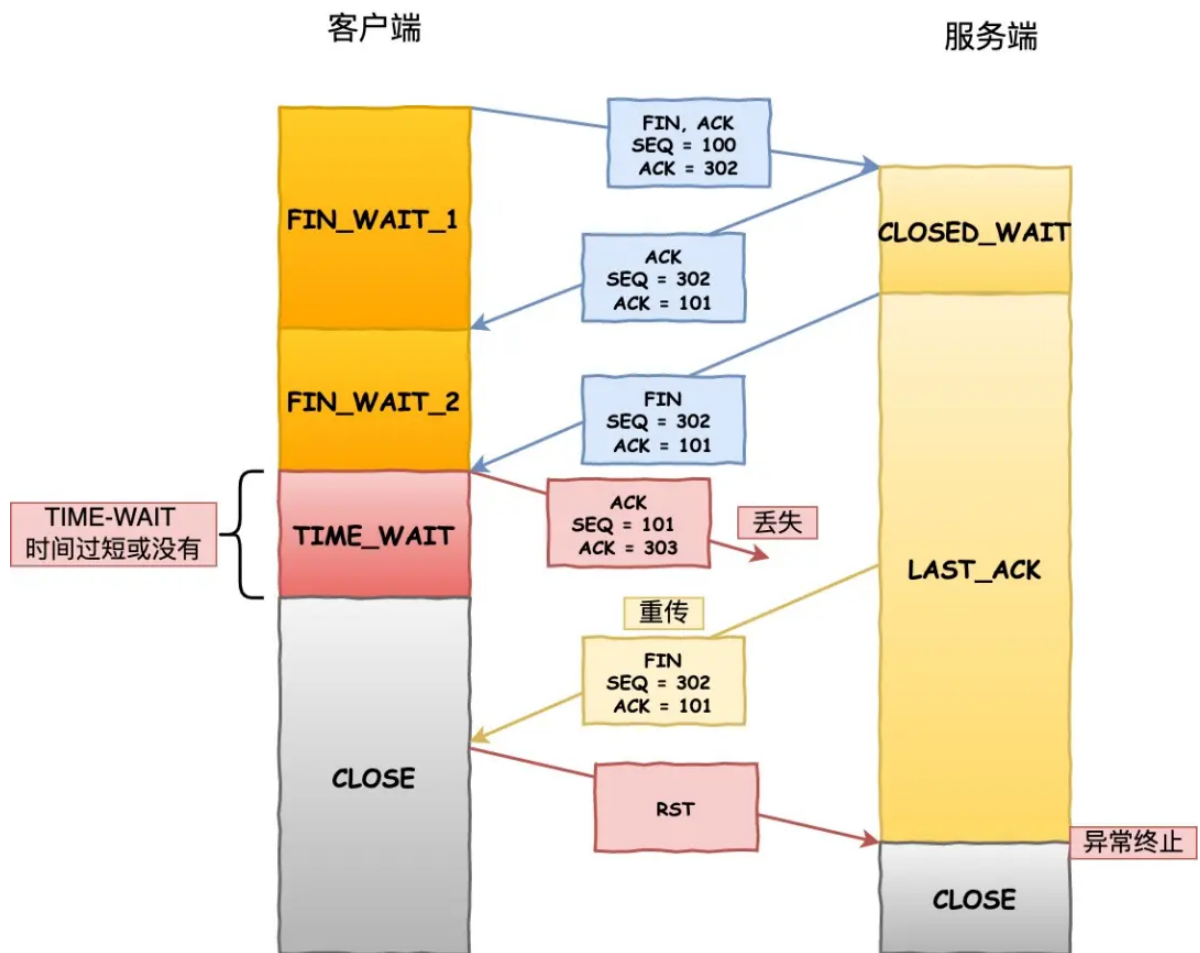
如上图：

- 服务端在关闭连接之前发送的 **SEQ = 301** 报文，被网络延迟了。
- 接着，服务端以相同的四元组重新打开了新连接，前面被延迟的 **SEQ = 301** 这时抵达了客户端，而且该数据报文的序列号刚好在客户端接收窗口内，因此客户端会正常接收这个数据报文，但是这个数据报文是上一个连接残留下来的，这样就产生数据错乱等严重的问题。

为了防止历史连接中的数据，被后面相同四元组的连接错误的接收，因此 TCP 设计了 **TIME_WAIT** 状态，状态会持续 **2MSL** 时长，这个时间足以让两个方向上的数据包都被丢弃，使得原来连接的数据包在网络中都自然消失，再出现的数据包一定都是新建立连接所产生的。

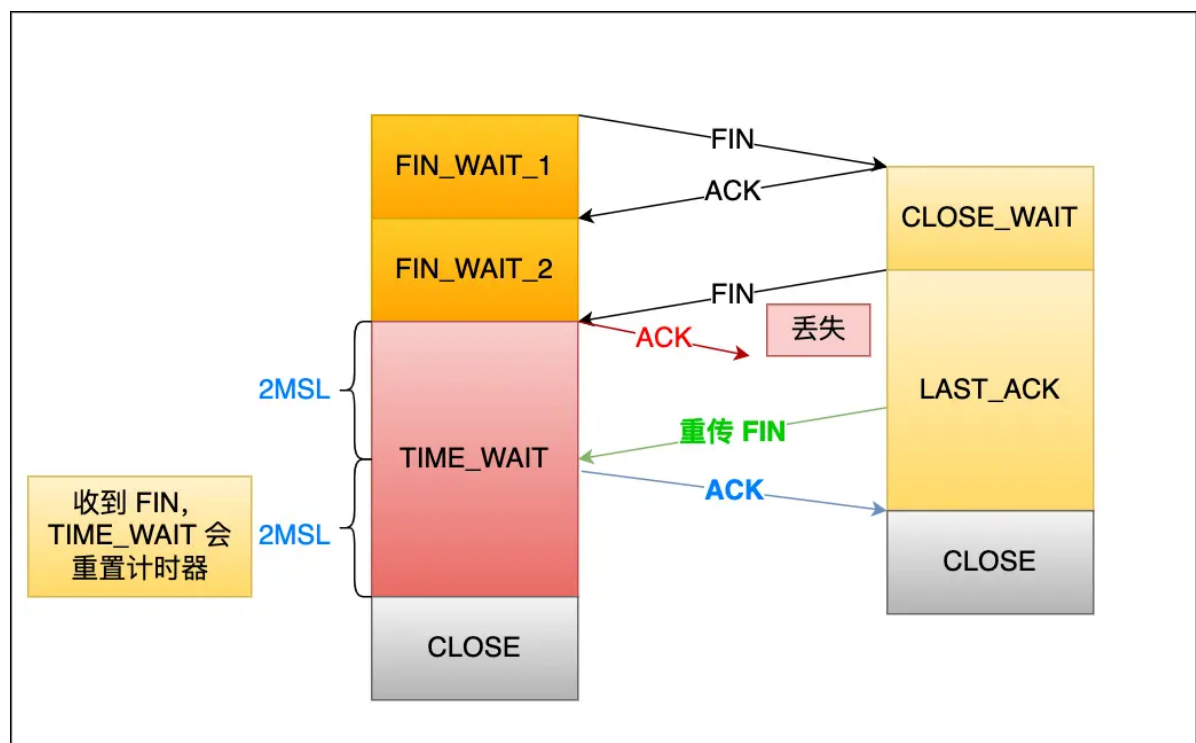
原因二：保证「被动关闭连接」的一方，能被正确的关闭

如果客户端（主动关闭方）最后一次 **ACK** 报文（第四次挥手）在网络中丢失了，那么按照 TCP 可靠性原则，服务端（被动关闭方）会重发 **FIN** 报文。假设客户端没有 **TIME_WAIT** 状态，而是在发完最后一次回 **ACK** 报文就直接进入 **CLOSE** 状态，如果该 **ACK** 报文丢失了，服务端则重传的 **FIN** 报文，而这时客户端已经进入到关闭状态了，在收到服务端重传的 **FIN** 报文后，就会回 **RST** 报文。



服务端收到这个 RST 并将其解释为一个错误 (Connection reset by peer)，这对于一个可靠的协议来说不是一个优雅的终止方式。

为了防止这种情况出现，客户端必须等待足够长的时间，确保服务端能够收到 ACK，如果服务端没有收到 ACK，那么就会触发 TCP 重传机制，服务端会重新发送一个 FIN，这样一去一来刚好两个 MSL 的时间。



客户端在收到服务端重传的 FIN 报文时，TIME_WAIT 状态的等待时间，会重置回 2MSL。

